# Лабораторна робота №3. Аналітичні запити та звітність

## Мета та очікувані результати

- Мета: перетворити дані власної предметної області на показники - спершу запитами до робочої схеми роботи №2, потім через окремий шар сховища. Робота показує, скільки коштує запит на великій таблиці і що з цим робить індекс, у якій формі зберігають дані для звітності (схема «зірка»), як їх туди переносить завантаження, яке витримує повторний запуск, і як показники подають дашбордом без написання запитів. Зірка і завантаження цієї роботи - останній крок реляційної частини курсу: робота №4 переносить частину даних у нереляційні системи.
- Після виконання роботи студент може: записати запит, відповідь на який - число, з групуванням, умовою на рядки у WHERE і умовою на групи в HAVING, і пояснити різницю логічним порядком виконання; порахувати всередині групи рядки, що задовольняють умову, виразом CASE; порівняти рядок із підсумком підзапитом у WHERE і в FROM; показати у звіті групу, у якої немає жодного рядка в таблиці подій, і пояснити, чому `count(*)` дає для неї 1; записати запити з ранжуванням, накопичувальним підсумком і ковзним середнім і пояснити, чим їхній результат відрізняється від групування; згенерувати таблицю подій приблизно на 100 тисяч рядків, прочитати `EXPLAIN ANALYZE` до і після створення індексу міграцією і назвати, чим за індекс платять; пояснити на плані виконання, чому індекс не прискорює аналітичний звіт; спроєктувати схему «зірка» своєї області - назвати зерно таблиці фактів словами, відрізнити показник від атрибута виміру, обґрунтувати склад двох вимірів - і створити її міграцією; записати завантаження трьома командами `INSERT ... SELECT` і довести числами, що повторний запуск не змінює показників; відбракувати брудні рядки джерела до і після перетворення типів і пояснити, чому саме в такому порядку; побудувати три показники і подати їх дашбордом у Looker Studio.
- Інструменти: pgAdmin із середовищем PostgreSQL роботи №1; Looker Studio - новий інструмент роботи, вебзастосунок для дашборду; Mermaid - діаграма схеми «зірка»; Git - здача.
- Що свідомо не входить у роботу: підзапити з EXISTS і конструкція WITH; рамки вікна, відмінні від ROWS; види індексів, індекси на кілька стовпців і на вирази; оцінки вартості в плані виконання; схема «сніжинка», повільно змінювані виміри, OLAP-куб, колонкові аналітичні СУБД, матеріалізовані подання і планувальники завдань. Нереляційне зберігання - робота №4.

## Словник термінів роботи

Нових термінів робота не вводить: усі терміни введено на лекціях 1-8. Приклади взято з прокату велосипедів; у звіті відповідні приклади наводяться для власної предметної області.

- **Предметна область** - обмежена частина реального світу, про яку система зберігає дані: її об’єкти, події та правила. Приклад: питання, на які відповідають показники цієї роботи, ставить замовник тієї самої області, для якої створено схему в роботі №1. Повторення з лекції 1.
- **Наскрізний проєкт** - формат практикуму: усі п’ять лабораторних робіт виконуються на одній предметній області, і кожна наступна робота розширює результат попередньої. Приклад: таблиця фактів цієї роботи наповнюється з тих самих рядків, які додав скрипт наповнення роботи №2. Повторення з лекції 1.
- **Дані** та **інформація** - зафіксовані окремі записи про факти і ті самі записи, поміщені в контекст та оброблені так, що відповідають на запитання «скільки», «де», «коли». Приклад: 10 рядків таблиці `rentals` - дані; картка «виручка за березень - 1110 грн» на дашборді - інформація. Повторення з лекції 1.
- **Факт** (запис) - одиничне твердження про те, що відбулося в конкретний момент. Приклад: «12 березня о 10:15 на пункті №2 видано велосипед B-12 за 250 грн» - один рядок таблиці фактів. Повторення з лекції 1.
- **Репозиторій** (Git) - каталог проєкту під контролем версій, у якому зберігається історія всіх змін; у курсі - єдиний спосіб здачі робіт. Приклад: окремі коміти для запитів, досліду з індексом, зірки і завантаження показують, у якому порядку виконано роботу. Повторення з лекції 1.
- **Система керування базами даних** (СУБД) - програмний комплекс, який зберігає дані, надає доступ до них, перевіряє правила й обслуговує одночасну роботу багатьох користувачів. Приклад: PostgreSQL сама обирає, читати всю таблицю видач чи шукати рядки за індексом; у файлі запиту про це немає жодного слова. Повторення з лекції 2.
- **Контейнер** і **файл опису середовища** - запущений екземпляр образу і текстовий файл, за яким його запускають однією командою. Приклад: перед початком роботи `docker compose up -d` у корені репозиторію піднімає той самий контейнер PostgreSQL, що й у роботах №1 і 2. Повторення з лекції 2.
- **Схема даних** - опис структури бази: які є таблиці, які в них стовпці, якого типу і які правила діють. Приклад: після цієї роботи в базі `dbsa` дві групи таблиць - робоча схема роботи №1 і таблиці сховища з префіксами `dim_` і `fact_`. Повторення з лекції 2.
- **Таблиця**, **рядок**, **стовпець** - множина рядків однакової структури, один екземпляр події чи об’єкта і одна його характеристика. Приклад: у таблиці `fact_rentals` один рядок - одна видача, а стовпець `rental_price` - її ціна. Повторення з лекції 2.
- **Тип даних** - обмеження на множину допустимих значень стовпця. Приклад: у проміжній таблиці всі стовпці мають тип `text`, тому вона приймає рядок «31.02.2026 08:00», який стовпець типу `timestamp` не прийняв би. Повторення з лекцій 2 і 5.
- **Обмеження** (constraint) - правило, записане в схемі й перевірене самою СУБД. Приклад: складений первинний ключ таблиці фактів не дає повторному завантаженню записати ту саму видачу двічі. Повторення з лекції 2.
- **Ідемпотентність** - властивість операції, за якої її повторне виконання не змінює результат. Приклад: після другого запуску завантаження в таблиці фактів ті самі 10 рядків і та сама сума 1170.00 грн. Повторення з лекцій 2, 5 і 8.
- **Первинний ключ** - той потенційний ключ, який обрано основним засобом ідентифікації; він обов’язковий і унікальний. Приклад: первинний ключ `fact_rentals` - пара «велосипед і час

видачі», та сама, що й у таблиці `rentals`. Повторення з лекції 3.

- **Природний ключ** і **сурогатний (штучний) ключ** - ключ із реальних даних предметної області і службовий ідентифікатор без змісту, створений системою. Приклад: ключ виміру пунктів - сурогатний `station_key`, а назва пункту лишається описовим стовпцем з обмеженням UNIQUE. Повторення з лекцій 3 і 8.
- **Зовнішній ключ** - атрибут, значення якого має відповідати первинному ключу рядка в іншій таблиці. Приклад: `fact_rentals.station_key` посилається на `dim_stations`, тому записати у факт пункт, якого немає у вимірі, неможливо. Повторення з лекції 3.
- **Посилальна цілісність** - вимога, щоб зовнішній ключ указував на рядок, який справді існує. Приклад: саме вона задає порядок команд завантаження - спершу виміри, потім факт. Повторення з лекцій 3 і 8.
- **Значення NULL** - позначка «значення відсутнє або невідоме», а не нуль і не порожній рядок. Приклад: у видачі, що триває, `rental_minutes` лишається NULL, тому середню тривалість рахують лише за поверненими видачами. Повторення з лекції 3.
- **Фізична модель** - опис того, як дані реально зберігаються в конкретній СУБД: типи, індекси, розміщення. Приклад: індекс за зовнішнім ключем змінює фізичну модель, а таблиці, стовпці й текст запиту лишаються тими самими. Повторення з лекцій 3 і 7.
- **Складений ключ** - первинний ключ із двох і більше атрибутів. Приклад: `ON CONFLICT (inventory_number, rented_at)` у команді завантаження факту перелічує обидва стовпці ключа. Повторення з лекції 4.
- **Нормалізація** і **третя нормальна форма** (3NF) - послідовне розкладання таблиць, щоб усунути аномалії, і форма, у якій кожен неключовий атрибут залежить від ключа і нічого, крім ключа. Приклад: робоча схема роботи №1 у 3NF, тому виручку в розрізі міста доводиться збирати з'єднанням кількох таблиць. Повторення з лекції 4.
- **Денормалізація** - свідоме повернення надмірності заради швидкості читання, з усвідомленням і контролем ризику неузгодженості. Приклад: місто записано в кожному рядку `dim_stations`; у сховищі це правильно, бо вимір цілком перезаписує завантаження, а не редагує користувач. Повторення з лекцій 4 і 8.
- **Похідне (обчислюване) значення** - значення, яке можна отримати з інших даних. Приклад: тривалість видачі у хвилинах у робочій базі не зберігають, а в таблиці фактів записують один раз під час завантаження, бо потрібна вона в кожному звіті. Повторення з лекцій 4 і 8.
- **SQL** (мова структурованих запитів) - мова, якою описують структуру реляційної бази, змінюють дані та отримують їх. Приклад: усі три кроки завантаження цієї роботи - команди SQL, а не програма. Повторення з лекції 5.
- **Декларативна мова** - мова, у якій описують потрібний результат, а спосіб його отримання обирає система. Приклад: текст запиту про видачі клієнта до і після створення індексу однаковий, а плани виконання різні. Повторення з лекції 5.
- **CREATE TABLE** і **DROP TABLE** - команди, які додають таблицю до схеми і видаляють її разом з усіма даними. Приклад: міграція зірки створює виміри раніше за факт, а скасовує у зворотному порядку - спершу `DROP TABLE fact_rentals`. Повторення з лекції 5.
- **Стовпець ідентичності** (identity column) - стовпець, значення якого СУБД видає сама за лічильником; так реалізують сурогатний ключ. Приклад: `station_key integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY` дає чотирьом пунктам прокату значення 1, 2, 3, 4. Повторення з лекцій 5 і 8.

- **Файл міграції** (migration) і **зворотна міграція** (down migration) - файл із командами однієї зміни схеми і файл, який точно її скасовує; команди у файлі down ідуть у зворотному порядку. Приклад: таблиці сховища додає окрема міграція, а не правка міграції 001. Повторення з лекції 5.
- **INSERT** (додати) - команда, яка додає рядки до таблиці; замість переліку рядків після VALUES можна записати запит SELECT, і тоді додаються всі рядки його результату. Приклад: `INSERT INTO fact_rentals (...) SELECT ... FROM rentals AS r JOIN dim_stations AS d ...`. Повторення з лекцій 5 і 7.
- **ON CONFLICT DO NOTHING** - доповнення до INSERT: рядок, який порушив би первинний ключ чи обмеження UNIQUE, мовчки пропускається. Приклад: повторне завантаження виміру дат повідомляє `INSERT 0 0`, бо всі 59 дат уже є. Повторення з лекцій 5 і 8.
- **UPSERT** (INSERT ... ON CONFLICT DO UPDATE) - доповнення до INSERT: якщо рядок із таким ключем уже є, його оновлюють значеннями нового рядка, доступними під іменем EXCLUDED. Приклад: видача, повернена після завантаження, наступного разу не дублюється, а оновлюється - `is_returned` стає істиною. Повторення з лекцій 5 і 8.
- **BEGIN**, **ROLLBACK** - команди, які відкривають явну транзакцію і скасовують усі її зміни. Приклад: `EXPLAIN ANALYZE` для команди UPDATE виконують лише між BEGIN і ROLLBACK, бо він справді змінює дані. Повторення з лекцій 5 і 7.
- **Запит** (query) - команда SELECT, яка повертає дані у вигляді таблиці-результату і нічого в базі не змінює. Приклад: жоден із запитів кроків 1 і 2 не змінює рядків таблиці подій. Повторення з лекцій 6 і 7.
- **SELECT**, **FROM**, **WHERE**, **ORDER BY** - команда вибірки і її частини: звідки брати рядки, яку умову вони мають задовольняти і в якому порядку їх показати. Приклад: `WHERE r.rented_at >= '2026-03-01' AND r.rented_at < '2026-04-01'` лишає березневі видачі ще до того, як їх розкладено за пунктами. Повторення з лекції 6.
- **З'єднання таблиць** (JOIN) і **ліве зовнішнє з'єднання** (LEFT JOIN) - частина запиту, яка ставить поруч рядки двох таблиць за умовою після ON, і його різновид, що лишає всі рядки лівої таблиці. Приклад: `dim_stations AS s LEFT JOIN fact_rentals AS f` показує у звіті і «Пункт №9», на якому видач не було. Повторення з лекції 6.
- **IS NULL, IS NOT NULL** - предикати перевірки на відсутність значення; єдиний спосіб знайти рядки з NULL. Приклад: `WHERE d.date_key IS NULL` після лівого з'єднання із виміром дат лишає рядки джерела з датою, якої не існує. Повторення з лекції 6.
- **Псевдонім** (alias) - коротке ім'я таблиці чи стовпця, чинне в межах одного запиту; пишеться англійською. Приклад: `count(*) AS rental_count` дає стовпцю результату назву, яку потім видно в Looker Studio. Повторення з лекцій 6 і 7.
- **Логічний порядок виконання запиту** - порядок, у якому СУБД опрацьовує частини SELECT: FROM і JOIN, WHERE, GROUP BY, HAVING, SELECT разом з віконними функціями, ORDER BY, LIMIT. Приклад: саме через нього умову з агрегатною функцією не можна записати у WHERE. Повторення з лекцій 6 і 7.
- **Агрегатна функція** (aggregate function) - функція, яка з багатьох рядків обчислює одне значення: `count`, `sum`, `avg`, `min`, `max`; значення NULL вона пропускає. Приклад: `sum(r.rental_price)` за десятьма видачами дає 1170.00. Повторення з лекції 7.
- **COUNT(*) і COUNT(стовпець)** - `count(*)` рахує рядки, `count(стовпець)` - лише рядки, у яких значення стовпця не NULL. Приклад: після лівого з'єднання для «Пункту №9»

`count(*)` дає 1, а `count(r.inventory_number)` - 0. Повторення з лекції 7.

- **round** (округлити) - функція, яка округлює число до вказаної кількості знаків після коми. Приклад: `round(avg(r.rental_price), 2)` показує 117.00 замість 117.0000000000000000. Повторення з лекції 7.
- **GROUP BY** (групувати за) - частина SELECT, яка розкладає рядки на групи з однаковими значеннями вказаних стовпців; у списку вибірки поруч з агрегатами стоять лише стовпці групування. Приклад: `GROUP BY r.station_name` над видачами повертає по рядку на пункт, де були видачі. Повторення з лекції 7.
- **HAVING** (за умови) - частина SELECT, яка лишає лише групи, для яких істинна умова з агрегатною функцією; опрацьовується після GROUP BY. Приклад: `HAVING sum(r.rental_price) > 350` лишає з трьох пунктів один. Повторення з лекції 7.
- **CASE** (умовний вираз) - вираз, який перевіряє умови WHEN по черзі і повертає значення THEN першої істинної; без ELSE повертає NULL. Приклад: `count(CASE WHEN r.returned_at IS NULL THEN 1 END)` рахує в кожній групі лише видачі, що тривають. Повторення з лекцій 7 і 8.
- **Підзапит** (subquery) - запит SELECT у дужках усередині іншого запиту: як значення в умові WHERE, як список для предиката IN або як таблиця у FROM, де він отримує псевдонім. Приклад: `WHERE b.condition_rating < (SELECT avg(condition_rating) FROM bikes)` повертає велосипеди, стан яких гірший за середній. Повторення з лекцій 7 і 8.
- **Віконна функція** (window function) - функція, яка для кожного рядка обчислює значення за набором пов'язаних з ним рядків - вікном - і не згортає рядків у групи; записується зі словом OVER. Приклад: `count(*) OVER (PARTITION BY r.station_name)` ставить поруч з кожною з 10 видач кількість видач її пункту, і рядків лишається 10. Повторення з лекції 7.
- **OVER, PARTITION BY** - OVER задає вікно: PARTITION BY ділить рядки на частини так само, як GROUP BY на групи, але рядки лишаються; ORDER BY усередині OVER задає їхній порядок у частині. Приклад: `rank() OVER (PARTITION BY r.station_name ORDER BY count(*) DESC)` - місце велосипеда серед велосипедів того самого пункту. Повторення з лекції 7.
- **Функції ранжування** (`rank`, `dense_rank`, `row_number`) - віконні функції, які нумерують рядки в порядку ORDER BY усередині OVER: `rank` дає однаковим значенням те саме місце і пропускає наступні номери, `dense_rank` - без пропусків, `row_number` - без повторів і в довільному порядку для однакових значень. Приклад: велосипеди B-12, B-31 і B-44 мають по дві видачі і ділять друге місце. Повторення з лекції 7.
- **Накопичувальний підсумок** (running total) - сума від першого рядка до поточного в заданому порядку; записується `sum(...) OVER (ORDER BY ...)`. Приклад: сума цін видач березня день за днем - 430, 680, 930, 1110 грн. Повторення з лекції 7.
- **Ковзне середнє** (moving average) - середнє за кількома останніми рядками, яке зсувається разом із поточним рядком; кількість рядків задає рамка `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. Рамка рахує рядки, а не календарні дні. Приклад: середня кількість видач за три останні дні з видачами - 4.00, 2.50, 2.00, 1.67. Повторення з лекції 7.
- **CAST** (привести тип) - перетворення значення в інший тип даних: `CAST(значення AS тип)`. Приклад: `CAST(r.rented_at AS date)` дає ключ виміру дат. Повторення з лекцій 7 і 8.
- **generate_series** (генерувати ряд) - функція PostgreSQL, яка повертає ряд значень як таблицю з одним стовпцем: цілі числа від першого аргументу до другого або дати із заданим кроком. Приклад: `generate_series(1, 100000) AS n` дає номери для скрипта

генерації, а `generate_series(timestamp '2026-02-01', timestamp '2026-03-31', interval '1 day')` - 59 дат для виміру. Повторення з лекцій 7 і 8.

- **План виконання** (query plan) - спосіб виконання запиту, обраний СУБД: як читати кожну таблицю і в якому порядку виконувати кроки. Приклад: для того самого запиту видач клієнта PostgreSQL без індексу обирає читання всієї таблиці, а з індексом - пошук за індексом. Повторення з лекцій 7 і 8.
- **EXPLAIN, EXPLAIN ANALYZE** - EXPLAIN показує план виконання, не виконуючи запиту; EXPLAIN ANALYZE виконує його і додає фактичну кількість рядків і час виконання (`Execution Time`). Приклад: у звіті наводять рядок плану і `Execution Time` до і після створення індексу. Повторення з лекцій 7 і 8.
- **Послідовне сканування** (Seq Scan) - план, у якому СУБД читає всі рядки таблиці підряд і перевіряє умову для кожного; `Rows Removed by Filter` показує, скільки рядків прочитано даремно. PostgreSQL може розділити таке читання між кількома процесами - тоді в плані `Parallel Seq Scan`. Приклад: для аналітичного звіту це не помилка плану, а єдиний розумний вибір - рядки потрібні всі. Повторення з лекцій 7 і 8.
- **Індекс** (index) - додаткова впорядкована структура, яку СУБД зберігає поруч із таблицею, щоб знаходити рядки за значенням стовпця, не переглядаючи всієї таблиці; він займає місце і сповільнює зміни даних. Для PRIMARY KEY і UNIQUE PostgreSQL створює індекс сама, для REFERENCES - ні. Приклад: індекс за номером клієнта знаходить 10 видач серед 100 тисяч. Повторення з лекцій 7 і 8.
- **CREATE INDEX, DROP INDEX** - команди, які створюють і видаляють індекс; як і будь-яка зміна схеми, оформлюються міграцією з файлами up і down. Приклад: `CREATE INDEX rentals_customer_number_idx ON rentals (customer_number);`. Повторення з лекції 7.
- **Сканування за індексом** (Index Scan, Bitmap Index Scan) - план, у якому СУБД знаходить потрібні рядки в індексі і читає з таблиці лише їх; ознака використання індексу - його назва в плані. Приклад: `Bitmap Index Scan on rentals_customer_number_idx`. Повторення з лекції 7.
- **Транзакційна система** (OLTP) - програма й база даних, які обслуговують щоденні дії: багато коротких операцій, кожна читає й змінює кілька рядків, відповідь потрібна негайно. Приклад: каса прокату записує видачу і читає дані одного клієнта. Уведено оглядово в лекції 1, повністю - у лекції 8.
- **Аналітичне навантаження** (OLAP) - запити, які читають багато рядків, щоб обчислити небагато чисел; вони не змінюють даних, виконуються рідше і довше. Приклад: звіт «видачі й сума за кожен день і пункт» читає всі рядки таблиці видач і повертає кілька сотень. Повторення з лекції 8.
- **Сховище даних** (data warehouse) - окрема база, у яку дані переносять з робочих систем для звітності й аналізу. Приклад: у цій роботі таблиці сховища створюють у тій самій базі `dbsa` поруч із робочими - так кожен крок завантаження вміщується в одну команду SQL. Уведено оглядово в лекції 1, повністю - у лекції 8.
- **Актуальність даних** (data freshness) - наскільки дані сховища відстають від робочої бази; визначається тим, як часто виконується завантаження. Приклад: якщо завантаження нічне, дашборд відповідає на питання «скільки заробили вчора», а не «скільки в касі зараз». Повторення з лекції 8.

- **Показник** (міра, measure) - число в таблиці фактів, яке має сенс додавати чи усереднювати за будь-яким розрізом. Приклад: ціна видачі - показник, а інвентарний номер велосипеда - ні, хоча теж записаний у рядку. Повторення з лекції 8.
- **Дашборд** (dashboard) - один екран, на якому зібрано кілька показників у різних розрізах і який читають без написання запитів. Приклад: дашборд прокату - картка виручки за місяць, стовпчикова діаграма за пунктами і часовий ряд за днями. Повторення з лекції 8.
- **BI-інструмент** (business intelligence) - програма, яка сама складає запити до підготовленої таблиці, коли користувач обирає показник і розріз, і малює результат діаграмою. Приклад: Looker Studio складає GROUP BY сам, коли розріз перетягують на вісь діаграми. Повторення з лекції 8.
- **Схема «зірка»** (star schema) - форма таблиць сховища: одна таблиця фактів у центрі і кілька таблиць вимірів навколо неї, кожна з яких посилається на факт одним ключем. Приклад: `fact_rentals` у центрі, `dim_stations` і `dim_dates` навколо; з'єднань у запиті рівно стільки, скільки вимірів у питанні. Повторення з лекції 8.
- **Таблиця фактів** (fact table) - таблиця сховища, у якій один рядок описує одну подію і містить показники цієї події та ключі вимірів. Приклад: `fact_rentals` - 10 рядків, по одному на видачу. Повторення з лекції 8.
- **Таблиця виміру** (dimension table) - таблиця сховища з описом того, у якому розрізі дивляться на показники; містить небагато рядків і багато описових стовпців. Приклад: `dim_stations` описує пункт - назву і місто. Повторення з лекції 8.
- **Зерно таблиці фактів** (grain) - відповідь на питання «що описує один рядок таблиці фактів»; обирається до того, як записано перший стовпець, і однакове для всіх рядків. Приклад: зерно `fact_rentals` - одна видача; зерно «день і пункт» дало б на тих самих даних у сотні разів менше рядків, але питання «який велосипед найпопулярніший» стало б неможливим. Повторення з лекції 8.
- **Вимір дат** (date dimension) - таблиця виміру з рядком на кожну календарну дату періоду і стовпцями, за якими групують: рік, місяць, день тижня, ознака вихідного. Приклад: `dim_dates` за лютий і березень 2026 року - 59 рядків. Повторення з лекції 8.
- **ETL** (extract - transform - load) - процес перенесення даних із робочих систем у сховище трьома кроками: витягнути сирі дані, перетворити їх до потрібної форми й перевірити, завантажити в таблиці сховища. Приклад: у цій роботі всі три кроки записані командами `INSERT ... SELECT`. Повторення з лекції 8.
- **Проміжна таблиця** (staging) - таблиця, у яку сирі дані завантажують як є, усіма стовпцями типу `text`, щоб перевірити їх, перш ніж перетворювати типи. Приклад: `stage_rentals` приймає «31.02.2026 08:00» без помилки, бо це просто текст. Повторення з лекції 8.
- **Брудні дані** (dirty data) - дані з джерела, яке не перевіряло правил: пропуски, дублікати, значення не того типу чи формату, суперечності між стовпцями. Приклад: у вивантаженні з електронної таблиці ціна записана як «60 грн», той самий рядок повторено двічі, а повернення позначено раніше за видачу. Повторення з лекції 8.
- **Відбраковані рядки** (rejects) - рядки джерела, які не пройшли перевірку і які завантаження відкладає в окрему таблицю разом із причиною, а не відкидає мовчки. Приклад: у `stage_rejects` записано номер рядка у файлі і причина - «такої дати не існує». Повторення з лекції 8.

- **DISTINCT** (лише різні) - слово на початку списку вибірки, яке лишає по одному рядку з кожної групи однакових; порівнюються всі стовпці списку вибірки. Приклад: `SELECT DISTINCT` прибирає з вивантаження повний дублікат рядка. Повторення з лекції 8.
- **to_timestamp, to_char** - функції PostgreSQL, які переводять значення між текстом і датою за шаблоном формату. Приклад: `to_timestamp('12.03.2026 10:15', 'DD.MM.YYYY HH24:MI')` дає значення дати й часу, а на неіснуючій даті завершується помилкою `date/time field value out of range`. Повторення з лекції 8.
- **trim, initcap, replace, left** - функції PostgreSQL для нормалізації тексту на кроці перетворення: обрізати пробіли, зробити з великої літери, замінити підрядок, узяти кілька символів зліва. Приклад: `initcap(trim(' пункт №11 '))` дає `Пункт №11`. Повторення з лекції 8.
- **nullif** (перетворити на NULL) - функція, яка повертає NULL, якщо значення дорівнює вказаному, і саме значення в іншому разі. Приклад: `nullif(s.returned_raw, '')` перетворює порожню клітинку файлу на NULL - «велосипед ще не повернуто». Повторення з лекції 8.
- **EXTRACT** (витягти частину) - функція, яка повертає складову дати, часу або проміжку часу. Приклад: `EXTRACT(isodow FROM d)` дає номер дня тижня для виміру дат, а `EXTRACT(epoch FROM r.returned_at - r.rented_at) / 60` - тривалість видачі у хвилинах. Повторення з лекції 8.

## Зв’язок з лекціями

Роботу виконують після лекції 8 на схемі роботи №1, наповненій даними роботи №2. Завдання для самостійної роботи лекцій 7 і 8 - чернетки кроків цієї роботи: їх доопрацьовують за вимогами нижче, а не створюють заново.

| Лекція | Що потрібно для роботи | Чернетка із завдання для самостійної роботи |
|---|---|---|
| 5 | `INSERT ... SELECT`, ON CONFLICT, стовпець ідентичності, міграції up і down | - |
| 6 | SELECT, WHERE, з’єднання таблиць, ліве з’єднання з IS NULL, псевдоніми | - |
| 7 | агрегатні функції, GROUP BY і HAVING, CASE, підзапити, віконні функції, `generate_series`, індекс, EXPLAIN ANALYZE | п’ять запитів на показники (завдання 1); запит про групу без рядків (завдання 2); три запити з віконними функціями (завдання 3); скрипт генерації і дослід з індексом (завдання 4) |
| 8 | OLTP і OLAP, сховище даних, схема «зірка», зерно, виміри, ETL, брудні дані, ідемпотентне завантаження, показники й дашборд | таблиця «робоча база - сховище» (завдання 1); зірка з двома вимірами (завдання 2); міграція зірки (завдання 3); три команди завантаження (завдання 4); зіпсована копія скрипта наповнення (завдання 5); три показники і дашборд (завдання 6) |

## Підготовка середовища

Робота виконується в середовищі робіт №1 і 2; з програм додається лише браузер для Looker Studio.

1. Запустити Docker Desktop (Windows, macOS) і дочекатися стану «Engine running»; у Linux має працювати служба Docker.
2. У терміналі (у Windows - PowerShell або Windows Terminal) перейти в корінь репозиторію і виконати:

   ```
   docker compose up -d
   docker compose ps
   ```

   Контейнер має бути в стані `running`.
3. У pgAdmin відкрити Query Tool для бази `dbsa` і виконати скрипт підрахунку рядків (`lab-02/count-rows.sql`) роботи №2: у таблицях мають бути рядки скрипта наповнення. Якщо база порожня, схему відтворюють файлами `*.up.sql` за зростанням номерів і виконують `seed-data.sql` роботи №2.
4. Перевірити, що дані роботи №2 придатні для показників: у таблиці подій є щонайменше 10 рядків, вони охоплюють не менше двох місяців і в них є принаймні три різні дні. Якщо рядків для показників замало, скрипт наповнення роботи №2 доповнюють у каталозі роботи №2, а зміну фіксують окремим комітом.
5. Перевірити доступ до Looker Studio: відкрити в браузері https://lookerstudio.google.com і увійти обліковим записом Google. Програм встановлювати не потрібно. Якщо облікового запису Google немає або сервіс недоступний, до викладача звертаються до початку роботи, а не в день здачі.
6. Для кроків 3 і 10 знадобляться окремі бази: `dbsa_big` для заміру часу і `dbsa_check` для повного прогону. Створюють їх у pgAdmin так само, як у роботі №2: Databases - Create - Database.

Якщо середовище не запускається, діють поради розділу «Типові помилки та як їх уникнути» роботи №1 (`lab-01.pdf`).

## Порядок виконання

### Крок 1. Запити на показники

- Створити в репозиторії каталог `lab-03` і в ньому файл `lab-03/analytic-queries.sql`. Усі запити цього кроку виконують у базі `dbsa`.
- Сформулювати словами п’ять питань замовника, відповідь на які - число, а не перелік рядків: кількість, сума, середнє, частка.

- Разом п’ять запитів використовують: GROUP BY - щонайменше в двох, причому в одному з них умова на рядки у WHERE поєднана з умовою на групи в HAVING; CASE усередині агрегатної функції; підзапит у WHERE; підзапит у FROM.
- Перед кожним запитом - коментар із номером, питанням словами, кількістю рядків результату і самим числом. Кожне число перевіряють вручну за рядками скрипта наповнення роботи №2: запит, результат якого нема з чим звірити, до звіту не подають.

```
-- Запит 2. Які пункти за березень набрали понад 350 грн?
-- Рядків: 1 - «Пункт №2», 4 видачі на 620.00.
SELECT r.station_name,
       count(*) AS rental_count,
       sum(r.rental_price) AS price_sum
FROM rentals AS r
WHERE r.rented_at >= '2026-03-01' AND r.rented_at < '2026-04-01'
GROUP BY r.station_name
HAVING sum(r.rental_price) > 350
ORDER BY price_sum DESC;
```

```
-- Запит 3. Скільки видач кожного пункту ще тривають?
-- Рядків: 3 - Пункт №2: 4 і 1, Пункт №4: 3 і 1, Пункт №7: 3 і 0.
SELECT r.station_name,
       count(*) AS rental_count,
       count(CASE WHEN r.returned_at IS NULL THEN 1 END) AS active_count
FROM rentals AS r
GROUP BY r.station_name
ORDER BY r.station_name;
```

- Окремо записати шостий запит - про групу, у якої немає жодного рядка в таблиці подій: групи беруть із таблиці, де є всі групи, лівим з’єднанням, а рахують стовпець правої таблиці, а не `*`.

```
-- Запит 6. Скільки видач на кожному пункті, разом із пунктами без жодної видачі?
-- Рядків: 4 - «Пункт №9» з нулем.
SELECT s.station_name,
       count(r.inventory_number) AS rental_count
FROM stations AS s
LEFT JOIN rentals AS r ON r.station_name = s.station_name
GROUP BY s.station_name
ORDER BY rental_count DESC, s.station_name;
```

- Для запиту з HAVING записати у звіт, що зміниться, якщо ту саму умову перенести у WHERE, і чому; для запиту 6 - чому `count(*)` дав би для порожньої групи 1.

Результат: файл `analytic-queries.sql` і розділ 1 звіту - таблиця «питання - використані конструкції - результат» і два пояснення.

### Крок 2. Три запити з віконними функціями

- Створити файл `lab-03/window-queries.sql` і записати в нього три запити до робочих таблиць бази `dbsa`: ранжування, накопичувальний підсумок, ковзне середнє.
- Ранжування: упорядкувати об'єкти області за кількістю або сумою подій. У даних має бути нічия - щонайменше два об'єкти з однаковим значенням. У звіті вказати, які місця дають нічиїм `rank`, `dense_rank` і `row_number` і яку з трьох функцій обрано для рейтингу.

```
-- Запит 1. Місце велосипеда за кількістю видач.
-- Рядків: 5. B-40 - перше місце (3 видачі); B-12, B-31 і B-44 ділять друге (по 2); B-27 -
п'яте.
SELECT r.inventory_number,
       count(*) AS rental_count,
       rank() OVER (ORDER BY count(*) DESC) AS rank_place
FROM rentals AS r
GROUP BY r.inventory_number
ORDER BY rank_place, r.inventory_number;
```

- Накопичувальний підсумок за днями: підзапит у FROM групує події за днем ( `CAST(...)` лишає від дати й часу саму дату), а зовнішній запит додає `sum(...) OVER (ORDER BY ...)`.

```
-- Запит 2. Як день за днем зростала сума за березень.
-- Рядків: 4 - 430.00, 680.00, 930.00, 1110.00.
SELECT d.rental_day,
       d.price_sum,
       sum(d.price_sum) OVER (ORDER BY d.rental_day) AS running_total
FROM (SELECT CAST(r.rented_at AS date) AS rental_day,
             sum(r.rental_price) AS price_sum
      FROM rentals AS r
      WHERE r.rented_at >= '2026-03-01' AND r.rented_at < '2026-04-01'
      GROUP BY CAST(r.rented_at AS date)) AS d
ORDER BY d.rental_day;
```

- Ковзне середнє за трьома рядками - рамка `ROWS BETWEEN 2 PRECEDING AND CURRENT ROW`. Для кожного значення результату у звіті вказати, які саме дні до нього ввійшли; якщо в періоді є дні без подій, окремо пояснити, що рамка рахує рядки, а не календарні дні.

```sql
-- Запит 3. Середня кількість видач за три останні дні з видачами.
-- Рядків: 4 - 4.00, 2.50, 2.00, 1.67. Значення 2.00 пораховано за 12, 13 і 21 березня.
SELECT d.rental_day,
       d.rental_count,
       round(avg(d.rental_count) OVER (ORDER BY d.rental_day
                                       ROWS BETWEEN 2 PRECEDING AND CURRENT ROW), 2) AS
moving_avg
FROM (SELECT CAST(r.rented_at AS date) AS rental_day,
             count(*) AS rental_count
      FROM rentals AS r
      WHERE r.rented_at >= '2026-03-01' AND r.rented_at < '2026-04-01'
      GROUP BY CAST(r.rented_at AS date)) AS d
ORDER BY d.rental_day;
```

- Якщо в стовпці, за яким іде ORDER BY усередині OVER, трапляються однакові значення, до порядку додають стовпці ключа - інакше рядки з однаковим значенням отримають той самий накопичувальний підсумок.
- У звіті пояснити на одному з трьох запитів, чим результат віконної функції відрізняється від результату GROUP BY: скільки рядків повертає кожен варіант і що видно лише у варіанті з вікном.

Результат: файл `window-queries.sql` і розділ 2 звіту - три запити з результатами, пояснення рамки ковзного середнього і порівняння з групуванням.

### Крок 3. Велика таблиця і дослід з індексом

Дослід виконують в окремій базі: сто тисяч згенерованих рядків не повинні потрапити в робочу базу `dbsa`.

- Створити базу `dbsa_big` і виконати в ній усі файли `*.up.sql` робіт №1 і 2 за зростанням номерів.
- Створити файл `lab-03/generate-data.sql` - скрипт генерації приблизно 100 тисяч рядків таблиці подій разом із рядками таблиць, на які вона посилається. Значення будують з номера рядка `n`, який дає `generate_series`; зразок оформлення - скрипт генерації для прокату (`lab-03/generate-sample.sql`).

```
-- Фрагмент generate-data.sql: 100 тисяч видач по 10 на клієнта, з кроком 5 хвилин.
INSERT INTO rentals (inventory_number, rented_at, returned_at,
                     customer_number, station_name, tariff_code, rental_price)
SELECT 'В-' || (5001 + n % 500),
       timestamp '2025-01-01 08:00' + n * interval '5 minutes',
       timestamp '2025-01-01 09:00' + n * interval '5 minutes',
       'К-' || (10001 + n % 10000),
       'Пункт №7',
       'Г1',
       60
FROM generate_series(1, 100000) AS n;
```

- Згенеровані значення не повинні збігатися з рядками скрипта наповнення роботи №2: інакше команда завершиться помилкою первинного ключа. Виконати скрипт у базі `dbsa_big` і записати кількість рядків у таблиці подій.
- Обрати запит, який шукає рядки за зовнішнім ключем і повертає їх небагато - десятки зі ста тисяч, - і виміряти його без індексу:

```
EXPLAIN ANALYZE
SELECT * FROM rentals WHERE customer_number = 'К-10042';
```

  Запит виконують тричі і беруть типове значення `Execution Time`: перший запуск зазвичай повільніший. Записати рядок плану (`Seq Scan on ...`), значення `Rows Removed by Filter` і час.

- Створити індекс міграцією в каталозі `lab-03/migrations` з наступним вільним номером (після робіт №1 і 2 це зазвичай 003). Індекс - об'єкт схеми, тому в діалозі pgAdmin його не створюють.

```
-- 003_add_customer_number_index.up.sql
-- Індекс за зовнішнім ключем: запит видач клієнта читає всю таблицю,
-- бо для REFERENCES PostgreSQL індексу не створює.

CREATE INDEX rentals_customer_number_idx ON rentals (customer_number);
```

```
-- 003_add_customer_number_index.down.sql
-- Скасування 003_add_customer_number_index.up.sql.

DROP INDEX rentals_customer_number_idx;
```

- Виконати файл up у базі `dbsa_big` і повторити замір тричі. Ознака того, що індекс використано, - його назва в плані; `Index Scan` і пара `Bitmap Index Scan` з `Bitmap Heap Scan` однаково означають пошук за індексом.

- Заповнити таблицю «до / після»: план, скільки рядків таблиці перевірено, час виконання. Порівнюють співвідношення часу, а не абсолютні числа: вони залежать від комп'ютера.
- Ціна індексу: виконати скрипт розміру таблиць та індексів (`lab-03/table-size.sql`) і записати розмір таблиці подій і розмір створеного індексу. У звіті одним реченням назвати другу частину ціни - сповільнення кожної команди INSERT, UPDATE і DELETE, яка оновлює всі індекси таблиці.
- Перевірити міграцію циклом up - down - up у базі `dbsa_big`: після down назва індексу зникає з плану виконання, після повторного up з'являється знову.
- Виконати файл up міграції індексу і в базі `dbsa`, щоб схема робочої бази відповідала файлам репозиторію.
- Прогноз і перевірка: записати два запити до тієї самої таблиці, для яких індекс не спрацює, - умову з `LIKE` і шаблоном, що починається з `%`, і умову з `<>`, якій відповідає майже кожен рядок. Для кожного записати очікування, виконати EXPLAIN і пояснити результат.

Результат: файл `generate-data.sql`, пара файлів міграції індексу і розділ 3 звіту - таблиця «до / після», розміри таблиці та індексу, два запити з поясненням, чому індекс їм не допоміг.

### Крок 4. Чому аналітику виносять з робочої бази

- У базі `dbsa_big` виконати `EXPLAIN ANALYZE` для аналітичного звіту: кількість і сума подій за кожен день і кожен об'єкт розрізу.

```
EXPLAIN ANALYZE
SELECT CAST(r.rented_at AS date) AS rental_day,
       r.station_name,
       count(*) AS rental_count,
       sum(r.rental_price) AS price_sum
FROM rentals AS r
GROUP BY CAST(r.rented_at AS date), r.station_name
ORDER BY rental_day, r.station_name;
```

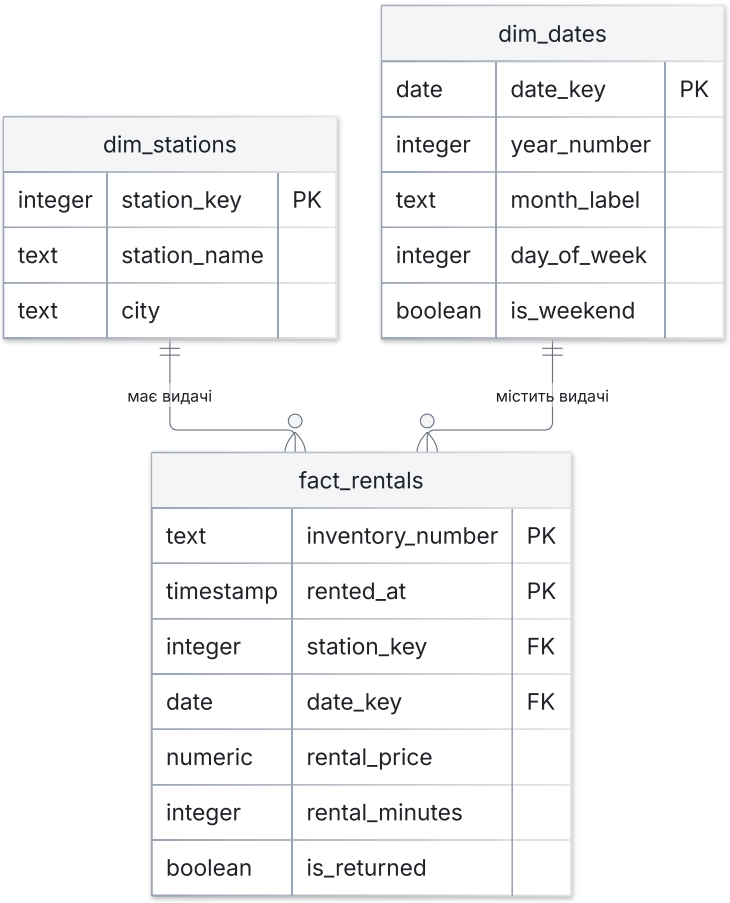
- Записати план і час виконання поруч із планом і часом запиту за зовнішнім ключем із кроку 3. Пояснити у звіті, чому індекс прискорив один запит у багато разів і не прискорив другий зовсім.
- Заповнити таблицю «робоча база - сховище»: по два питання замовника кожного типу - про «зараз» і про період, - скільки рядків читає запит, як часто він виконується і що станеться, якщо дані застаріли на годину.
- Сформулювати, з якою частотою має завантажуватися сховище власної області, і обґрунтувати це саме тими питаннями, які записано в таблиці.

Результат: розділ 4 звіту - два плани виконання з поясненням, заповнена таблиця і обґрунтована частота завантаження.

### Крок 5. Проєктування схеми «зірка»

- Записати словами зерно таблиці фактів: що описує один її рядок. Зерно формулюють до того, як записано перший стовпець, і воно однакове для всіх рядків.
- Перелічити показники таблиці фактів - не менше двох - і для кожного пояснити, чому його осмислено додавати або усереднювати за будь-яким розрізом. Якщо в таблиці подій робочої схеми немає жодного числового стовпця, показник беруть похідним - тривалість події у хвилинах чи днях, кількість днів до завершення; додавати стовпець міграцією в робочу схему заради цього не потрібно.
- Обрати два виміри. Перший - розріз власної області (пункт, категорія, працівник, клієнт) із сурогатним ключем і природним ключем як описовим стовпцем з обмеженням UNIQUE. Другий - вимір дат: рядок на кожну календарну дату періоду даних і стовпці, за якими групують.
- Стовпці, які не стали ні показником, ні атрибутом обраних вимірів, у таблиці фактів лишаються як є (наприклад, стовпці природного ключа події) або не переносяться взагалі; рішення записують одним реченням.
- Намалювати схему діаграмою `erDiagram` у Mermaid: назви таблиць і стовпців англійською, точно як у міграції; типи PostgreSQL без параметрів; кожен атрибут складеного ключа позначається `PK`, зовнішні ключі - `FK`.

```
erDiagram
    dim_stations ||--o{ fact_rentals : "має видачі"
    dim_dates ||--o{ fact_rentals : "містить видачі"
    dim_stations {
        integer station_key PK
        text station_name
        text city
    }
    dim_dates {
        date date_key PK
        integer year_number
        text month_label
        integer day_of_week
        boolean is_weekend
    }
    fact_rentals {
        text inventory_number PK
        timestamp rented_at PK
        integer station_key FK
        date date_key FK
        numeric rental_price
        integer rental_minutes
        boolean is_returned
    }
```




- Окремим абзацом описати денормалізацію виміру: який стовпець повторюється в його рядках, чому це не загрожує аномаліями оновлення і чим це відрізняється від ситуації робочої бази.

Результат: розділ 5 звіту - зерно словами, перелік показників з обґрунтуванням, склад двох вимірів, діаграма і абзац про денормалізацію.

### Крок 6. Міграція схеми «зірка»

- Створити в `lab-03/migrations` пару файлів міграції з наступним вільним номером (зазвичай 004). Таблиці сховища створюють у тій самій базі `dbsa` поруч із робочими - так кожен крок завантаження вміщується в одну команду SQL; префікси `dim_` і `fact_` показують, що це інший шар схеми.
- У файлі up спершу створюють виміри, потім таблицю фактів; у файлі down - зворотний порядок.
- Ключ виміру-розрізу - стовпець ідентичності; його природний ключ отримує обмеження UNIQUE, бо саме на ньому триматиметься повторне завантаження. Ключ таблиці фактів - природний ключ події з робочої таблиці, а не стовпець ідентичності.

```sql
-- 004_create_star_schema.up.sql
-- Таблиці сховища: виміри пунктів і дат та таблиця фактів видач.

CREATE TABLE dim_stations (
    station_key  integer GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    station_name text NOT NULL UNIQUE,
    city         text NOT NULL
);

CREATE TABLE dim_dates (
    date_key    date PRIMARY KEY,
    year_number integer NOT NULL,
    month_label text NOT NULL,
    day_of_week integer NOT NULL,
    is_weekend  boolean NOT NULL
);

CREATE TABLE fact_rentals (
    inventory_number text NOT NULL,
    rented_at        timestamp NOT NULL,
    station_key      integer NOT NULL REFERENCES dim_stations (station_key) ON DELETE
RESTRICT,
    date_key         date NOT NULL REFERENCES dim_dates (date_key) ON DELETE RESTRICT,
    rental_price     numeric(8, 2) NOT NULL,
    rental_minutes   integer,
    is_returned      boolean NOT NULL,
    PRIMARY KEY (inventory_number, rented_at)
);
```

```sql
-- 004_create_star_schema.down.sql
-- Скасування 004_create_star_schema.up.sql: спершу факт, потім виміри.

DROP TABLE fact_rentals;
DROP TABLE dim_dates;
DROP TABLE dim_stations;
```

- Виконати файл up у базі `dbsa` і перевірити цикл up - down - up скриптом перевірки схеми (`lab-01/check-schema.sql`): після up у результаті є три нові таблиці з ключами, після down їх немає, після повторного up вони знову на місці. Робочі таблиці цикл не зачіпає.

Результат: пара файлів міграції зірки і розділ 6 звіту - результат скрипта перевірки схеми для трьох таблиць сховища і підтвердження циклу.

### Крок 7. Завантаження: три команди і повторний запуск

- Створити файл `lab-03/load-star.sql` із трьома командами `INSERT ... SELECT`: два виміри, потім таблиця фактів. Порядок задає посилальна цілісність: зовнішній ключ факту не пустить рядок, поки у вимірі немає відповідного рядка.
- Кожна команда має доповнення ON CONFLICT на ключі, який робить повторний запуск безпечним: для виміру - його природний ключ, для факту - складений ключ події.

```
-- load-star.sql, крок 1: вимір пунктів.
INSERT INTO dim_stations (station_name, city)
SELECT s.station_name, s.city
FROM stations AS s
ON CONFLICT (station_name) DO UPDATE
SET city = EXCLUDED.city;

-- Крок 2: вимір дат за період даних.
INSERT INTO dim_dates (date_key, year_number, month_label, day_of_week, is_weekend)
SELECT CAST(d AS date),
       CAST(EXTRACT(year FROM d) AS integer),
       to_char(d, 'YYYY-MM'),
       CAST(EXTRACT(isodow FROM d) AS integer),
       EXTRACT(isodow FROM d) >= 6
FROM generate_series(timestamp '2026-02-01', timestamp '2026-03-31', interval '1 day') AS
d
ON CONFLICT (date_key) DO NOTHING;

-- Крок 3: таблиця фактів.
INSERT INTO fact_rentals (inventory_number, rented_at, station_key, date_key,
                          rental_price, rental_minutes, is_returned)
SELECT r.inventory_number,
       r.rented_at,
       s.station_key,
       CAST(r.rented_at AS date),
       r.rental_price,
       CAST(EXTRACT(epoch FROM r.returned_at - r.rented_at) / 60 AS integer),
       r.returned_at IS NOT NULL
FROM rentals AS r
JOIN dim_stations AS s ON s.station_name = r.station_name
ON CONFLICT (inventory_number, rented_at) DO UPDATE
SET station_key    = EXCLUDED.station_key,
    date_key       = EXCLUDED.date_key,
    rental_price   = EXCLUDED.rental_price,
    rental_minutes = EXCLUDED.rental_minutes,
    is_returned    = EXCLUDED.is_returned;
```

- Період виміру дат беруть за даними власної області: перша й остання дата події. Значення, яких немає в робочій таблиці, у факт не переносять і нічим не замінюють: `rental_minutes` для події, що триває, лишається NULL.
- Виконати завантаження і записати результат: кількість рядків у кожній із трьох таблиць і суму головного показника. Потім виконати той самий файл удруге і записати ті самі числа ще раз - вони мають збігтися.
- Повідомлення СУБД після другого запуску не є доказом: `ON CONFLICT DO UPDATE` рахує й оновлені рядки, тому обидва рази повідомлення однакове. Перевіряють `count(*)` і суму показника, а не повідомлення.
- Показати на одному рядку, що завантаження оновлює, а не дублює: змінити в робочій таблиці одне значення (позначити подію завершеною або виправити число), виконати завантаження втретє і показати, що кількість рядків факту не змінилася, а значення в ньому стало новим.

Результат: файл `load-star.sql` і розділ 7 звіту - таблиця «до першого запуску - після першого - після другого» з кількістю рядків і сумою показника, повідомлення СУБД обох запусків і результат досліду з оновленням рядка.

### Крок 8. Брудні дані та відбраковані рядки

- Джерело цього кроку - вивантаження нового підрозділу, який вів облік в електронній таблиці: у ньому немає обмежень схеми, тому перевірку робить завантаження.
- Створити пару файлів міграції з наступним вільним номером (зазвичай 005) для двох таблиць: проміжної таблиці, у якій усі стовпці мають тип `text` і є стовпець із номером рядка у файлі, і таблиці відбракованих рядків із номером рядка й причиною.

```
-- 005_create_staging.up.sql
-- Проміжний шар завантаження: сирі рядки джерела і відбраковані рядки з причиною.

CREATE TABLE stage_rentals (
    source_row           integer PRIMARY KEY,
    inventory_number_raw text,
    rented_raw           text,
    returned_raw         text,
    station_raw          text,
    price_raw            text
);

CREATE TABLE stage_rejects (
    source_row    integer PRIMARY KEY,
    reject_reason text NOT NULL
);
```

- Створити файл `lab-03/stage-source.sql`: копія кількох рядків власного скрипта наповнення, записаних у проміжну таблицю як текст і зіпсованих чотирма способами

заняття - пропуск обов’язкового значення, повний дублікат рядка, значення не того типу чи формату, суперечність між стовпцями. Разом у файлі 6-8 рядків, з яких частина коректна.

```
-- stage-source.sql: вивантаження пункту №11 як текст, разом із помилками джерела.
INSERT INTO stage_rentals (source_row, inventory_number_raw, rented_raw, returned_raw,
station_raw, price_raw)
VALUES (1, 'B-51', '12.03.2026 10:15', '12.03.2026 12:40', 'Пункт №11', '60 грн'),
       (2, 'B-52', '31.02.2026 08:00', '', 'Пункт №11', '60 грн'),
       (3, 'B-51', '12.03.2026 10:15', '12.03.2026 12:40', 'Пункт №11', '60 грн'),
       (4, 'B-53', '13.03.2026 09:00', '', 'пункт №11 ', ''),
       (5, 'B-54', '14.03.2026 18:00', '14.03.2026 08:00', 'Пункт №11', '60 грн'),
       (6, 'B-55', '15.03.2026 11:30', '', 'Пункт №11', '250 грн');
```

- Створити файл `lab-03/stage-load.sql` з двома частинами. Перша відбраковує те, що завалило б саме перетворення типів: дату, якої не існує, - звіркою з виміром дат, і незаповнене обов’язкове число. Рядки з причиною переносять у таблицю відбракованих, а не відкидають мовчки.

```
-- Частина 1. Відбракування до перетворення типів.
INSERT INTO stage_rejects (source_row, reject_reason)
SELECT s.source_row,
       CASE WHEN s.price_raw = '' THEN 'ціна не заповнена'
            ELSE 'такої дати не існує' END
FROM stage_rentals AS s
LEFT JOIN dim_dates AS d ON to_char(d.date_key, 'DD.MM.YYYY') = left(s.rented_raw, 10)
WHERE s.price_raw = '' OR d.date_key IS NULL
ON CONFLICT (source_row) DO NOTHING;
```

- Друга частина перетворює типи в підзапиті у FROM, а вже на перетворених значеннях прибирає дублікати словом DISTINCT і відсіює суперечності. Порівнювати дати на тексті проміжної таблиці не можна: для рядків із різних місяців результат буде неправильний.

```
-- Частина 2. Перетворення, перевірка суперечностей і завантаження у факт.
INSERT INTO fact_rentals (inventory_number, rented_at, station_key, date_key,
                          rental_price, rental_minutes, is_returned)
SELECT DISTINCT
       t.inventory_number,
       t.rented_at,
       s.station_key,
       CAST(t.rented_at AS date),
       t.rental_price,
       CAST(EXTRACT(epoch FROM t.returned_at - t.rented_at) / 60 AS integer),
       t.returned_at IS NOT NULL
FROM (SELECT st.inventory_number_raw AS inventory_number,
             to_timestamp(st.rented_raw, 'DD.MM.YYYY HH24:MI') AS rented_at,
             to_timestamp(nullif(st.returned_raw, ''), 'DD.MM.YYYY HH24:MI') AS
returned_at,
             initcap(trim(st.station_raw)) AS station_name,
             CAST(replace(st.price_raw, ' грн', '') AS numeric(8, 2)) AS rental_price
      FROM stage_rentals AS st
      LEFT JOIN stage_rejects AS rj ON rj.source_row = st.source_row
      WHERE rj.source_row IS NULL) AS t
JOIN dim_stations AS s ON s.station_name = t.station_name
WHERE t.returned_at IS NULL OR t.returned_at > t.rented_at
ON CONFLICT (inventory_number, rented_at) DO UPDATE
SET rental_price   = EXCLUDED.rental_price,
    rental_minutes = EXCLUDED.rental_minutes,
    is_returned    = EXCLUDED.is_returned;
```

- Рядки джерела посилаються на розріз, якого може ще не бути у вимірі: перед завантаженням його додають тією самою командою виміру з кроку 7 або окремим рядком у робочій таблиці - обраний спосіб записують у звіті.
- Виконати обидві частини і записати: скільки рядків було у файлі, скільки відбраковано і з якої причини, скільки прибрав DISTINCT, скільки відсіяла перевірка суперечностей, скільки рядків додано до таблиці фактів. Для кожної з чотирьох проблем указати обмеження робочої схеми (NOT NULL, UNIQUE або первинний ключ, тип стовпця, CHECK), яке не пустило б такий рядок у базу.
- Виконати `stage-load.sql` удруге і показати, що кількість рядків у таблиці фактів не змінилася.

Результат: пара файлів міграції проміжного шару, файли `stage-source.sql` і `stage-load.sql` і розділ 8 звіту - таблиця «рядок файлу - проблема - на якому кроці відбраковано - обмеження робочої схеми» і числа завантаження.

### Крок 9. Показники і дашборд

- Створити файл `lab-03/dashboard-queries.sql` із трьома запитами до таблиць сховища - по одному на елемент дашборду: підсумок за період; той самий показник у розрізі виміру-розрізу, разом із рядками, у яких подій не було; показник день за днем.

```sql
-- Показник 1. Виручка за місяць.
-- Рядків: 2 - 2026-02: 60.00, 2026-03: 1110.00.
SELECT d.month_label,
       sum(f.rental_price) AS revenue
FROM fact_rentals AS f
JOIN dim_dates AS d ON d.date_key = f.date_key
GROUP BY d.month_label
ORDER BY d.month_label;

-- Показник 2. Видачі за пунктами, разом із пунктами без видач.
-- Рядків: 4 - 4, 3, 3 і «Пункт №9» з нулем.
SELECT s.station_name,
       count(f.inventory_number) AS rental_count
FROM dim_stations AS s
LEFT JOIN fact_rentals AS f ON f.station_key = s.station_key
GROUP BY s.station_name
ORDER BY rental_count DESC, s.station_name;
```

- У звіті порівняти запит показника 2 із запитом 6 кроку 1: у зірці перелік груп бере на себе вимір, бо він містить усі значення розрізу за означенням.
- Кожен запит виконати в Query Tool і зберегти його результат у файл CSV кнопкою збереження результату (Download as CSV) на панелі результатів. Файли CSV зберігати в каталозі `lab-03`.
- У Looker Studio створити звіт, додати файли CSV як джерела даних (додавання джерела завантаженням файлу) і побудувати три елементи: картку показника, стовпчикову діаграму в розрізі виміру і часовий ряд за днями. Показник перетягують у поле показника, розріз - у поле розрізу; запит з групуванням інструмент складає сам.
- Зберегти знімок готового дашборду як `lab-03/dashboard.png`. Поруч зі знімком у звіті записати частоту завантаження сховища з кроку 4 - без неї числа дашборду читають як сьогоднішні.
- У звіті пояснити, чому дашборд будують на вивантаженні CSV, а не прямим підключенням до бази: PostgreSQL працює в контейнері на власному комп'ютері і з інтернету недоступний; на робочому місці це буде конектор до сервера сховища, а форма таблиць лишиться тією самою.

Результат: файл `dashboard-queries.sql`, файли CSV, знімок `dashboard.png` і розділ 9 звіту - три запити з результатами, знімок дашборду, частота завантаження і два пояснення.

### Крок 10. Повний прогін у порожній базі

- У pgAdmin створити порожню базу `dbsa_check` і відкрити для неї Query Tool.
- Виконати всі файли `*.up.sql` з каталогів `lab-01/migrations`, `lab-02/migrations` і `lab-03/migrations` за зростанням номерів, потім скрипт перевірки схеми (`lab-01/check-schema.sql`): у результаті є і робочі таблиці, і три таблиці сховища, і дві таблиці проміжного шару.
- Виконати файли down міграцій роботи №3 за спаданням номерів і знову скрипт перевірки схеми: таблиць сховища і проміжного шару немає, робочі таблиці лишилися. Потім виконати ті самі файли up ще раз - результат має збігатися з першим.
- Виконати `seed-data.sql` роботи №2, потім `load-star.sql`, потім скрипт підрахунку рядків (`lab-02/count-rows.sql`): кількість рядків у вимірах і таблиці фактів має збігатися з розділом 7 звіту.
- Виконати `load-star.sql` удруге і скрипт підрахунку рядків ще раз: числа ті самі.
- Виконати кожен запит із `analytic-queries.sql`, `window-queries.sql` і `dashboard-queries.sql`: результати мають збігатися з коментарями у файлах і зі звітом. Розбіжність означає, що в базі `dbsa` дані відрізняються від скрипта наповнення; її виправляють і повторюють прогін.
- Після перевірки бази `dbsa_check` і `dbsa_big` можна видалити; файли скриптів лишаються в репозиторії.

Результат: розділ 10 звіту.

## Структура каталогу і здача через Git

```
<репозиторій студента>/
├── README.md                                  опис предметної області (лекція 1)
├── compose.yaml                               файл опису середовища, спільний для робіт
├── lab-01/                                    робота №1
├── lab-02/                                    робота №2
└── lab-03/
    ├── README.md                              звіт за шаблоном
    ├── analytic-queries.sql                   запити на показники (крок 1)
    ├── window-queries.sql                     три віконні функції (крок 2)
    ├── generate-data.sql                      генерація великої таблиці (крок 3)
    ├── load-star.sql                          завантаження сховища (крок 7)
    ├── stage-source.sql                       сире вивантаження джерела (крок 8)
    ├── stage-load.sql                         відбракування і перетворення (крок 8)
    ├── dashboard-queries.sql                  три показники дашборду (крок 9)
    ├── dashboard.png                          знімок дашборду
    ├── *.csv                                  вивантаження для Looker Studio
    └── migrations/
        ├── 003_add_customer_number_index.up.sql
        ├── 003_add_customer_number_index.down.sql
        ├── 004_create_star_schema.up.sql
        ├── 004_create_star_schema.down.sql
        ├── 005_create_staging.up.sql
        └── 005_create_staging.down.sql
```

- Звіт - файл `lab-03/README.md`, створений зі шаблону звіту (`lab-03/report-template.md`). Діаграма зірки - блок `mermaid` у звіті; окремого файлу діаграми немає.
- Нумерація міграцій наскрізна для всіх робіт: номери 003, 004 і 005 наведено для випадку, коли робота №2 додала міграцію 002. Якщо в роботі №2 міграції не було, номери зсуваються на один.
- Закомічені файли міграції не редагують; кожна зміна схеми - нова пара файлів up і down з однаковим номером і описом.
- Скрипти генерації, завантаження і запитів міграціями не є і в каталог `migrations` не потрапляють.
- Знімок дашборду зберігається у форматі PNG; файли CSV - ті самі, що завантажено в Looker Studio.
- Усі SQL-файли і звіт зберігаються в кодуванні UTF-8.
- Робота виконується кількома змістовними комітами: щонайменше окремі коміти для запитів на показники, віконних функцій, досліду з індексом, зірки з міграцією, завантаження, брудних даних і дашборду. Повідомлення коміту описує зміну: «Додано міграцію схеми зірка і завантаження вимірів», а не «update».
- Історія комітів є частиною оцінювання: вона підтверджує самостійність роботи і те, що закомічені файли міграції не змінювалися.

- На перевірку подається стан репозиторію в основній гілці. Терміни здачі і спосіб повідомлення про готовність роботи оголошує викладач.

## Зміст звіту

Звіт `lab-03/README.md` містить розділи в такому порядку (усі вони вже є в шаблоні звіту (`lab-03/report-template.md`)):

1. Запити на показники - таблиця «питання - використані конструкції - результат», пояснення умови в HAVING і пояснення групи без рядків.
2. Віконні функції - три запити з результатами, дні, що ввійшли в рамку ковзного середнього, порівняння вікна з групуванням.
3. Дослід з індексом - кількість згенерованих рядків, таблиця «до / після», розміри таблиці та індексу, два запити, яким індекс не допоміг.
4. Два навантаження - плани виконання аналітичного звіту і запиту за ключем, таблиця «робоча база - сховище», обґрунтована частота завантаження.
5. Схема «зірка» - зерно словами, показники з обґрунтуванням, склад вимірів, діаграма Mermaid, абзац про денормалізацію.
6. Міграція зірки - посилання на файли, результат скрипта перевірки схеми і цикл up - down - up.
7. Завантаження - три команди, числа до першого запуску, після першого і після другого, дослід з оновленням рядка.
8. Брудні дані - таблиця «рядок файлу - проблема - на якому кроці відбраковано - обмеження робочої схеми» і числа завантаження.
9. Показники і дашборд - три запити з результатами, знімок дашборду, частота завантаження, пояснення про перелік груп у вимірі й про вивантаження CSV.
10. Повний прогін у порожній базі - результати кроку 10.

## Критерії приймання

- У каталозі `lab-03` є звіт з усіма розділами, файли `analytic-queries.sql`, `window-queries.sql`, `generate-data.sql`, `load-star.sql`, `stage-source.sql`, `stage-load.sql`, `dashboard-queries.sql`, знімок дашборду і файли міграцій індексу, зірки та проміжного шару.
- П’ять запитів кроку 1 разом використовують GROUP BY (щонайменше двічі, один раз - разом із WHERE і HAVING), CASE усередині агрегатної функції, підзапит у WHERE і підзапит у FROM; для кожного записано питання словами і перевірений вручну результат.
- Запит про групу без рядків побудовано лівим з’єднанням із рахуванням стовпця, і пояснено, чому `count(*)` дав би 1.
- Три запити з віконними функціями виконано: ранжування з нічиєю і поясненням різниці `rank`, `dense_rank` і `row_number`; накопичувальний підсумок; ковзне середнє з переліком днів, що ввійшли в рамку. Пояснено, чим результат вікна відрізняється від GROUP BY.

- Таблиця подій у базі заміру містить близько 100 тисяч рядків, згенерованих `generate_series`; заміри виконано тричі і взято типове значення.
- Таблиця «до / після» містить план, кількість перевірених рядків і час до і після індексу; індекс створено міграцією і проходить цикл up - down - up; названо обидві складові ціни індексу.
- Показано два запити, яким індекс не допомагає, з поясненням причини; для аналітичного звіту пояснено, чому послідовне сканування тут - правильний план.
- Таблиця «робоча база - сховище» заповнена питаннями власної області, і частота завантаження обґрунтована саме цими питаннями.
- Зерно таблиці фактів записано словами; кожен показник обґрунтовано ознакою «його осмислено додавати»; жоден атрибут виміру не потрапив у показники і навпаки.
- Вимір-розріз має сурогатний ключ і UNIQUE на природному ключі; вимір дат містить рядок на кожну дату періоду; таблиця фактів має первинний ключ з реальних даних, а не стовпець ідентичності.
- Діаграма Mermaid відповідає файлам міграції: ті самі назви, типи, ключі; діаграма відображається у вебінтерфейсі сервісу репозиторію без помилок.
- Міграції зірки і проміжного шару проходять цикл up - down - up; у файлі down таблиці видаляються у зворотному порядку.
- Завантаження записано трьома командами `INSERT ... SELECT` у правильному порядку, кожна з ON CONFLICT; після другого запуску кількість рядків і сума показника не змінилися, і в звіті наведено числа, а не повідомлення СУБД.
- Джерело брудних даних містить усі чотири проблеми; для кожної вказано, на якому кроці її відбраковано і яке обмеження робочої схеми не пустило б такий рядок; відбраковані рядки лежать у таблиці з причиною.
- Дашборд містить три елементи на трьох показниках; знімок є в репозиторії; поруч записано частоту завантаження.
- Назви таблиць, стовпців, псевдонімів, індексів і файлів міграції - англійською, малими літерами, через підкреслення, без лапок і транслітерації.
- Повний прогін у порожній базі проходить без помилок, а числа збігаються зі звітом.
- Історія Git складається з кількох змістовних комітів.
- На захисті студент відповідає на контрольні питання на прикладі власної бази і власного дашборду.

## Контрольні питання до захисту

1. Покажіть свій запит з HAVING. Що станеться, якщо перенести цю умову у WHERE, і чому?
2. Чому у вашому запиті про групу без подій використано `count` стовпця, а не `count(*)`?
3. Скільки рядків повертає ваш запит з віконною функцією і скільки повернув би той самий розріз через GROUP BY? Що видно лише у варіанті з вікном?
4. Два об’єкти у вашому рейтингу мають однакове значення. Які місця дадуть їм `rank`, `dense_rank` і `row_number` і яку функцію ви обрали?

5. За які саме дні пораховано третє значення вашого ковзного середнього? Чи можна назвати його середнім за три календарні дні?
6. Покажіть у своїй таблиці «до / після» рядок плану, за яким видно, що індекс використано. Чому для запиту з `LIKE '%...'` він не спрацював?
7. Чим ви платите за свій індекс? Чому не варто створити індекс для кожного стовпця таблиці?
8. Чому індекс за зовнішнім ключем прискорив ваш запит у багато разів і не прискорив аналітичний звіт?
9. Назвіть зерно своєї таблиці фактів. Що зникло б із ваших звітів, якби ви взяли зерно на рівень грубіше?
10. Чому обраний вами показник - це показник, а не атрибут виміру? Наведіть із власної схеми стовпець, який на показник схожий, але ним не є.
11. Чому ключ вашого виміру сурогатний, а ключ таблиці фактів - природний?
12. Ваш вимір містить стовпець, який повторюється в багатьох рядках. Чому це не порушення правил роботи №1?
13. У якому порядку виконуються три ваші команди завантаження і що станеться, якщо почати з таблиці фактів?
14. Обидва запуски завантаження повідомили однакове число доданих рядків. Чи означає це, що другий запуск додав рядки? Як ви це перевірили?
15. Який рядок вашого джерела відбраковано до перетворення типів, а який - після? Чому цей порядок не можна поміняти місцями?
16. У вашому джерелі є порожня клітинка, яку ви залишили, і порожня клітинка, через яку рядок відбраковано. Чим вони відрізняються?
17. Менеджер каже, що число на вашому дашборді не збігається з даними системи. Назвіть три можливі причини, спираючись на свою роботу.
18. Чому в запиті показника 2 перелік груп бере на себе вимір, а в запиті кроку 1 його довелося здобувати лівим з'єднанням з робочою таблицею?

## Типові помилки та як їх уникнути

- Запит завершується помилкою `column "..." must appear in the GROUP BY clause or be used in an aggregate function`. Протидія: поруч з агрегатними функціями в списку вибірки стоять лише стовпці групування; решту або прибирають, або додають у GROUP BY.
- Умову з агрегатною функцією записано у WHERE - помилка `aggregate functions are not allowed in WHERE`. Протидія: умова на рядки - у WHERE, умова на групу - у HAVING.
- `sum` від CASE без ELSE дає для групи без жодного збігу NULL замість 0. Протидія: рядки, що задовольняють умову, рахують через `count(CASE ... END)`; для сум дописують `ELSE 0`.
- Середнє по групах не враховує тих, у кого подій немає. Протидія: групи беруть із таблиці, де є всі групи, лівим з'єднанням, а рахують стовпець правої таблиці.
- Віконну функцію записано у WHERE - помилка `window functions are not allowed in WHERE`. Протидія: запит із віконною функцією стає підзапитом у FROM, а умова на її результат - у

зовнішньому запиті.

- Накопичувальний підсумок однаковий у кількох сусідніх рядків. Протидія: у стовпці ORDER BY усередині OVER є однакові значення; до порядку додають стовпці ключа.
- Ковзне середнє називають «середнім за три дні», хоча в періоді є дні без подій. Протидія: рамка ROWS рахує рядки; у звіті для кожного значення перелічують дні, що до нього ввійшли.
- Скрипт генерації запущено в базі `dbsa`, і робочі дані загубилися серед ста тисяч рядків. Протидія: генерацію виконують лише в базі `dbsa_big`; назву бази перевіряють у заголовку Query Tool.
- Скрипт генерації завершується помилкою первинного ключа. Протидія: згенеровані значення ключа не повинні збігатися з рядками скрипта наповнення роботи №2 - для генерації беруть інший діапазон номерів і дат.
- Індекс створено, а план показує `Seq Scan`. Протидія: перевірити, чи запит справді відбирає мало рядків; шаблон LIKE з `%` на початку, умова `<>` і стовпець, обгорнутий функцією, індексом за стовпцем не користуються.
- У плані немає слів `Index Scan`, а є `Bitmap Heap Scan`, і студент вирішує, що індекс не спрацював; або після кількох запусків генерації таблиця розрослася і план показує `Parallel Seq Scan`. Протидія: ознака використання індексу - його назва в плані; `Parallel Seq Scan` - те саме послідовне читання кількома процесами; перед заміром базу відтворюють циклом down - up і генерують дані один раз.
- Висновок зроблено за одним заміром. Протидія: кожен запит виконують тричі і беруть типове значення; порівнюють співвідношення «до / після», а не абсолютні числа.
- `EXPLAIN ANALYZE` для UPDATE чи DELETE змінив дані. Протидія: такі команди досліджують лише між BEGIN і ROLLBACK.
- Таблиці сховища створено в діалозі pgAdmin без міграції. Протидія: будь-яка зміна схеми - пара файлів up і down; цикл up - down - up перевіряє обидва.
- Файл down міграції зірки завершується помилкою `cannot drop table ... because other objects depend on it`. Протидія: у файлі down спершу видаляють таблицю фактів, потім виміри.
- Завантаження факту завершується помилкою `insert or update on table ... violates foreign key constraint`. Протидія: команди виконують у порядку «виміри, потім факт»; якщо порядок правильний, у вимірі немає рядка для значення розрізу - його додають тією самою командою виміру.
- Первинним ключем таблиці фактів зроблено стовпець ідентичності, і повторне завантаження тихо задвоїло рядки. Протидія: ключ таблиці фактів беруть той самий, що й у робочій таблиці події.
- Повторний запуск перевіряють за повідомленням СУБД. Протидія: `ON CONFLICT DO UPDATE` рахує й оновлені рядки, тому повідомлення однакові; порівнюють `count(*)` і суму показника до і після.
- Команда завантаження завершується помилкою `ON CONFLICT DO UPDATE command cannot affect row a second time`. Протидія: у вхідних рядках двічі трапився той самий ключ; повні дублікати прибирає DISTINCT, а рядки з однаковим ключем і різними значеннями відбраковують як суперечність джерела.

- Перетворення типів виконано першою командою, і один поганий рядок завалив усе завантаження помилкою `date/time field value out of range`. Протидія: сирі рядки спершу кладуть у проміжну таблицю з текстовими стовпцями і відбраковують до перетворення.
- Порівняння дат виконано на тексті проміжної таблиці, і рядок із суперечністю пройшов перевірку. Протидія: суперечності перевіряють після перетворення типів; перетворення обгортають підзапитом у FROM.
- Відбраковані рядки відкинуто мовчки умовою у WHERE. Протидія: кожен відбракований рядок потрапляє в таблицю відбракованих із номером рядка у файлі і причиною.
- Порожню клітинку вважають помилкою завжди, і події, що тривають, відбраковано. Протидія: розрізняють пропуск, який означає «події ще не сталося» (його перетворюють на NULL функцією `nullif`), і пропуск обов'язкового значення.
- Looker Studio не бачить бази під час спроби підключитися до `localhost`. Протидія: контейнер працює лише на власному комп'ютері; шлях роботи - вивантаження результату запиту в CSV і додавання джерела завантаженням файлу.
- Числа на дашборді відрізняються від чисел у звіті. Протидія: CSV вивантажують після останнього завантаження сховища; у звіті поруч із дашбордом записують, на який момент вивантажено дані.
- Українські підписи в CSV відображаються пошкодженими. Протидія: файл зберігають у кодуванні UTF-8; псевдоніми стовпців у запитах записують англійською, як і решта назв структури.
- Кількість рядків або сума показника у звіті не збігається з повторним виконанням. Протидія: числа фіксують одразу після завантаження, а перевіряють у порожній базі на кроці 10.

## Забезпечення роботи

- Скрипт розміру таблиць та індексів (`lab-03/table-size.sql`) - виконується в pgAdmin без змін, показує розмір кожної таблиці й індексу схеми.
- Скрипт генерації для прокату (`lab-03/generate-sample.sql`) - зразок оформлення генерації з `generate_series`; виконується лише після переписування під власну схему.
- Шаблон звіту (`lab-03/report-template.md`) - копіюється як `lab-03/README.md`.
- З попередніх робіт: файл опису середовища (`lab-01/compose.yaml`), скрипт перевірки схеми (`lab-01/check-schema.sql`) і скрипт підрахунку рядків (`lab-02/count-rows.sql`).
- Програмне забезпечення: Docker Desktop (Windows, macOS) або Docker Engine із Docker Compose (Linux), у Windows - WSL 2; образ `postgres:18.6`; pgAdmin 4 версії 9.17; браузер і обліковий запис Google для Looker Studio; Git.
- Зразки запитів, міграцій і команд завантаження - колоди слайдів лекції 7 і лекції 8.
- Роздатковий матеріал лекцій: шпаргалка «логічний порядок виконання з GROUP BY і HAVING» і бланк таблиці «до / після» (лекція 7); бланк таблиці «робоча база - сховище», чистий шаблон зірки з місцем для зерна й показників і знімок готового дашборду (лекція 8).

## Рекомендовані джерела

- Beaulieu A., розділи про групування й агрегування, підзапити, умовну логіку, аналітичні функції та індекси.
- Silberschatz A. та ін., розділ 3 (агрегатні функції і вкладені підзапити), розділ 5 (ранжування і віконні функції), розділ 14 (Indexing) і розділ 11 (Data Analytics) - сховище даних, схема «зірка», ETL.
- Kimball R., Ross M. The Data Warehouse Toolkit - розділ 1 (факти, виміри, зерно) і розділ 2 (базові прийоми); потребує перевірки.
- Берко А. Ю. та ін. Кн. 2, розділи про мову SQL.
- Офіційна документація PostgreSQL, підрозділи Aggregate Functions, Window Functions і Conditional Expressions, розділ Indexes і підрозділ Using EXPLAIN, підрозділи Data Type Formatting Functions і String Functions, опис команди INSERT (підрозділ ON CONFLICT Clause).
- Use The Index, Luke - розділи Anatomy of an SQL Index і The Where Clause.
- Довідка Google Looker Studio - розділи про додавання джерела даних завантаженням файлу і про типи діаграм (картка показника, стовпчикова діаграма, часовий ряд).